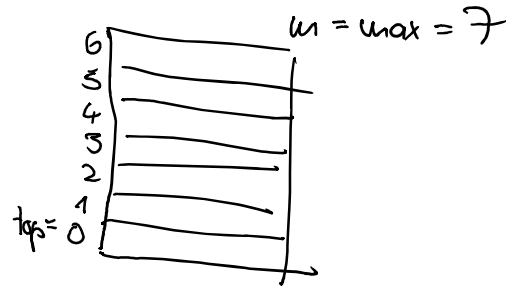


Implementierung der Datenstruktur Stack als Array:

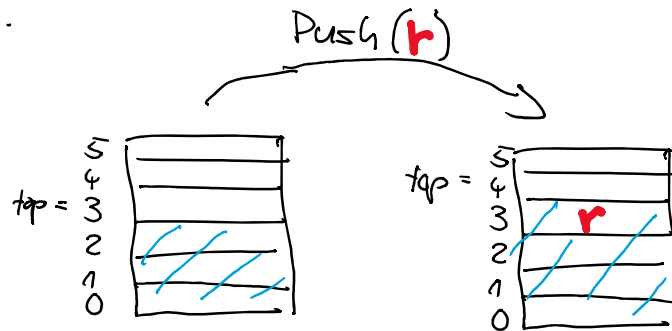
```
// Konstruktor: legt Speicherplatz für max Records an
// Setzt top auf 0 und m auf max
template <class Tr>
Stack<Tr>::Stack(unsigned max){
    m=max;
    a=new Tr[m];
    top=0;
}

// Destruktor löscht alle Einträge im Stack
template <class Tr>
Stack<Tr>::~~Stack(){
    delete a[];
}
```



Implementierung der Datenstruktur Stack als Array:

```
// oben im Stapel ablegen, Bedingung: Stapel nicht voll
template <class Tr>
void Stack<Tr>::Push(Tr r)
{
    if (!IsFull())
    {
        a[top]=r;
        top++;
    }
    else
        printf("Stack ist voll - Push() nicht möglich");
}
```



Grenzfälle:

Stack leer → alles ok

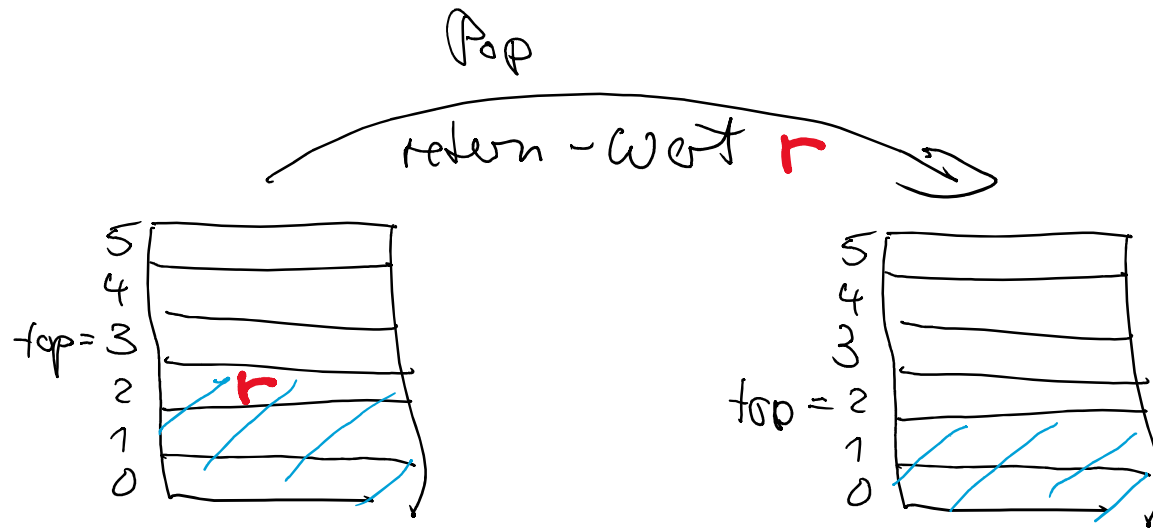
Stack voll → abgefangen durch IsFull()

Implementierung der Datenstruktur Stack als Array:

```
// obersten Record holen und entfernen,
// Bedingung: Stapel nicht leer
template <class Tr>
Tr Stack<Tr>::Pop()
{
    Tr r;
    if (!IsEmpty())
    {
        top--;
        r=a[top];
        return r;
    }
    else
        printf("Stack ist leer - Pop nicht möglich");
}
```

Algorithmen und Datenstrukturen SS2026

Prof. Dr. U. Göhner HS Kempten 25



Grenzfälle:

1. leerer Stack → Fehler abgefangen durch IsEmpty()
2. voller Stack → kein Problem